

Topic 4: The Ingestion Parser — Extracting Trajectories from FHIR Bundles

TOPIC OBJECTIVE

Before any multi-agent reasoning can take place, raw nested FHIR JSON must be parsed, stripped of metadata envelopes, and normalized into clean clinical vectors. In `src/parser.py`, we implement a zero-dependency parser that handles polymorphic inputs, normalizes dates across synthetic timelines, and chronologically sorts lab observations to establish baseline-to-acute clinical trajectories.

1. The Real-World Analogy: The Airport Luggage Sorting Hub

Imagine an international cargo jet lands and dumps thousands of mixed items onto a single chaotic conveyor belt. Mixed together are passenger suitcases (Patient data), fragile blood sample vials (Observations), prescription pill bottles (Medications), and doctor diagnostic letters (Conditions).

The Parser (`src/parser.py`) is the automated high-speed barcode scanner and robotic sorter:

- **Strips Packaging Fluff:** Discards HTTP status codes, fullUrl UUID wrappers, and system metadata that agents don't need.
- **Sorts into Dedicated Carts:** Places labs onto the Lab Cart, pills onto the Pharmacy Cart, and diagnoses onto the History Cart.
- **Lines Up the Timeline:** Organizes lab sample vials in chronological order from oldest to newest, so doctors can instantly spot whether kidney function is worsening over time.

2. Technical Design Decisions in `src/parser.py`

A. Polymorphic Ingestion (CLI & Web Unified)

The parser accepts file paths (str or `pathlib.Path`) or pre-loaded in-memory dictionaries. This enables offline CLI testing and real-time FastAPI web uploads (`/api/upload`) to share the exact same code path with zero overhead:

```
def parse_synthea_bundle(source: Union[str, Path, Dict[str, Any]]) -> Dict[str, Any]: if
    isinstance(source, (str, Path)): with open(source, "r", encoding="utf-8") as f: data =
    json.load(f) elif isinstance(source, dict): data = source
```

B. The Chronological Sorting Mechanism

A patient's FHIR bundle may contain lab tests in random order. To calculate **KDIGO Acute Kidney Injury**, we must know what the patient's creatinine was *before* (baseline) versus *today* (current presentation). On line 156, the parser sorts all observations by ISO-8601 timestamp:

```
# Sort labs chronologically so history/deltas are easy to trace raw_labs.sort(key=lambda x:
x.get("effective_datetime") or "")
```

3. Concrete Example: Arthur Morales (Input vs. Output)

Here is how Arthur Morales's raw FHIR bundle is transformed into the clean Python state dictionary:

Raw FHIR JSON (Input)	Normalized Parser Output (Python Dict)
<pre>{ 'resourceType': 'Observation', 'code': {'coding': [{'code': '2160-0'}]}, 'effectiveDateTime': '2026-09-08', 'valueQuantity': {'value': 1.0} } { 'resourceType': 'Observation', 'code': {'coding': [{'code': '2160-0'}]}, 'effectiveDateTime': '2026-09-12', 'valueQuantity': {'value': 2.4} }</pre>	<pre>'raw_labs': [{ 'name': 'Serum Creatinine', 'code': '2160-0', 'value': 1.0, 'effective_datetime': '2026-09-08' }, { 'name': 'Serum Creatinine', 'code': '2160-0', 'value': 2.4, 'effective_datetime': '2026-09-12' }]</pre>

KNOWLEDGE CHECK & NEXT STEP

1. Why is chronological sorting mandatory for AKI detection?

Because KDIGO AKI math compares baseline creatinine (oldest test) against acute presentation (newest test).
Unsorted labs lead to inverted ratios.

2. Why avoid third-party FHIR SDK libraries?

Following the Ponytail principle: Python's standard library json parses the exact 4 resources we need in 150 lines without bloated external dependencies.

3. Next Step in Topic 5: The Deterministic Rules Engine — Mathematical KDIGO AKI Staging and the Triple Whammy Interaction Matrix.